

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – ANALYTICAL PROBLEM SOLVING

Course Code:	CSA0309	Course Title:	Data Structures
CO Assessed:	CO2 – Manipulation (BL3, BL4)	Assessment:	Assessment Tool 1 – Analytical Problem Solving
Weightage:	30% of CIA	Total Marks:	100
CO2 Statement:	Implement linear data structures such as stacks, queues, and linked lists, and analyze the efficiency of linear and binary search techniques.		
Student Name:	K. Pujitha	Reg. No.:	192524149
Section / Batch:	2 nd year (AI&DS)	Date:	17/07/2026

Code of Conduct

I K. Pujitha (192524149) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: K. Pujitha

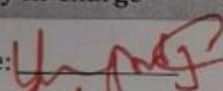
Instructions

- This test contains 80 Analytical problem-solving questions. Total: 100 Marks.
- When a student answer using an Analytical problem-solving, in evaluation the answer should be with following five requirements: Understanding of problem, select appropriate data structure, Design the solution, analyze, Clarity of representation.
- Each student should write 2 questions. No negative marking. Unanswered questions carry zero marks.
- No DTP printouts or electronic devices permitted. They should provide written materials.

Section / Topic	Focus	Q Nos	Marks
Linked Data Structure, Search Data Structure	List Operations, Binary Search	1	50
Stack Data Structure, Queue Data Structures	Stack Notations, Priority Queue	2	50
GRAND TOTAL:			100 Marks

Rubrics for Calculation

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Problem Understanding	20	Demonstrates complete and precise understanding; identifies all constraints and edge cases	Shows clear understanding with minor gaps	Partial understanding; misses some constraints	Misinterprets problem or lacks clarity
Data Structure Selection	20	Selects optimal data structure with strong justification and comparison	Selects appropriate structure with reasonable justification	Selects somewhat suitable structure with weak reasoning	Incorrect or no data structure selection
Algorithm Design	20	Designs efficient, logically sound, and well-structured algorithm	Algorithm is correct with minor inefficiencies	Algorithm is partially correct or lacks clarity	Algorithm is incorrect or missing
Accuracy of Solution	30	Error-free, well-structured, optimized code/solution	Minor errors but overall functional	Multiple errors; partially working	Non-functional or missing solution
Clarity	10	Exceptionally clear, well-organized, uses diagrams effectively	Clear and organized with minor issues	Somewhat organized but lacks clarity	Poorly organized and unclear

Faculty In-charge	Course Coordinator	Program Director
Signature: 	Signature: _____	Signature: _____
Date: <u>18/8/26</u>	Date: _____	Date: _____

Now a queue may be maintained by a circular array with $\text{MAXSIZE} = 6$. Observe that queue always occupies consecutive locations except when it occupies locations at the beginning and at the array. If the queue is viewed as a circular array, this means that it still occupies consecutive locations. The queue is empty when $\text{Count} = 0$. Perform the following ENQUEUE and DEQUEUE operations.

a) Initially queue is empty,

b) A, B, C are Enqueued

c) A is Dequeued

d) D, E, F are Enqueued

e) B & C are Dequeued

f) G is Enqueued

g) D & E are Dequeued

h) H & I are Enqueued.

i) F is Dequeued.

j) G & H are Dequeued.

k) I is Dequeued.

Demonstrate with a neat sketch of Front, Rear, and Count in a queue and Circular queue.

A. Implementation of Circular Queue using Array.

(MAXSIZE = 6)

Definition:

A queue is a linear data structure that follows the FIFO (First In First Out) principle. The first Element inserted into the queue is the first Element removed.

A Circular queue is an improved version of a linear queue in which the last position is connected to the first position.

Operations of Circular Queue:

Enqueue:

- Inserts an Element at the Rear
- Rear is updated using:

$$\text{Rear} = (\text{Rear} + 1) \bmod \text{MAXSIZE}.$$

Dequeue:

- Removes an Element from the front.
- Front is updating using:

$$\text{Front} = (\text{Front} + 1) \bmod \text{MAXSIZE}$$

Conditions:

Queue Empty

$$\text{Count} = 0$$

Queue full

Count = MAXSIZE

Initial Queue:

MAXSIZE = 6

INDEX : 0 1 2 3 4 5

Queue : [] [] [] [] [] []

Front = 0

Rear = -1

Count = 0

Step	operation	Queue Contents	Front	Rear	Count
a	Initially Empty	-	0	-1	0
b	Enqueue A	A	0	0	1
	Enqueue B	A B	0	1	2
	Enqueue C	A B C	0	2	3
c	Dequeue A	B C	1	2	2
d	Enqueue D	B C D	1	3	3
	Enqueue E	B C D E	1	4	4
	Enqueue F	B C D E F	1	5	5
e	Dequeue B, C	D E F	3	5	3
f	Enqueue G	D E F G	3	0	4

g	Dequeue D, E	F G	5	0	2
h	Enqueue H	F G H	5	1	3
.	Enqueue I	F G H I	5	2	4
i	Dequeue F	G H I	0	2	3
j	Dequeue G, H	I	2	2	1
k	Dequeue I	Empty	3	2	0

Circular Queue Diagram After step (h)

Array

Index : 0 1 2 3 4 5

 | G | H | I | | | F |

Front = 5

Rear = 2

Logical order:

Front $\rightarrow$ F $\rightarrow$ G $\rightarrow$ H $\rightarrow$ I $\leftarrow$ Rear.

- ② show the Linked list Configuration Resulting from each series of list operations using the list ADT. Assume that lists L₁, L₂ are Empty at the beginning of each series. Show

where the Current Position is in the list

[a] L1.append(10); L1.append(20); L1.append(15);

[b] L2.append(10); L2.append(20); L2.append(15);

L2.moveToStart(); L2.insert(39);

L2.next(); L2.insert(12);

A. Linked List Configuration using list ADT

Definition:

A linked list is a dynamic data structure consisting of nodes. Each node contains:

- Data
- Address

Basic List ADT Operations:

Operation	Description
append(x)	Inserts an Element at End of list
insert(x)	Inserts an Element at Current Position
moveToStart()	moves Current Pointer to first node
next()	moves Current Pointer to next node

(a) operations:

L1.append(10)

L1.append(20)

L1.append(15)

Step 1:

10 → NULL

Current

↓

10

Step 2:

10 → 20 → NULL

Current

↓

20

Step 3

10 → 20 → 15 → NULL

Current

↓

15

Final LI

Head

↓

10 → 20 → 15 → NULL

Current → 15.

(b) Operations:-

L2. append (10)

L2. append (20)

L2. append (15)

L2. moveToStart ()

L2. insert (39)

L2. next ()

4

19. Insert (12).

Step 1:

$10 \rightarrow 20 \rightarrow 15 \rightarrow \text{Null}$

Step 2:

move Current to start

Current

↓

~~$10 \rightarrow 20 \rightarrow 15 \rightarrow \text{Null}$~~

Step 3:

Insert 39 after Current

$10 \rightarrow 39 \rightarrow 20 \rightarrow 15 \rightarrow \text{NULL}$

Current

↓

10

Step 4:

move Current to next

$10 \rightarrow 39 \rightarrow 20 \rightarrow 15 \rightarrow \text{Null}$

Current

↓

~~39~~

Step 5:

Insert 12 after Current.

$10 \rightarrow 39 \rightarrow 12 \rightarrow 20 \rightarrow 15 \rightarrow \text{Null}$

Final 12 Configuration:

Head



10 → 39 → ~~12~~ → 20 → 15 → Null

Current



39.

Wgdy